Лабораторная работа №12

Задание 1.

```c
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_ARR_LEN    100

void print_array(int* arr, int length)
{
   for(int i = 0; i < length; ++i) printf("%d ", arr[i]);
   putchar('\n');
}

void insert_sort(int* arr, int length)
{
   int i_last_sorted_element = 0;

   for(int i = 0; i < length; ++i) {
      int index_to_insert = i;
      for(int j = 0; j <= i_last_sorted_element; ++j) {
         if(arr[j] >= arr[i]) {
            index_to_insert = j;
            break;
         }
      }

      int tmp_elem = arr[i];

      for(int k = i; k > index_to_insert; --k) {
         arr[k] = arr[k-1];
      }

      arr[index_to_insert] = tmp_elem;
      ++i_last_sorted_element;
   }
}

int main (void)
{
   // srand(time(NULL));
   int array_to_sort[MAX_ARR_LEN];

   for(int i = 0; i < MAX_ARR_LEN; ++i) {
```

```
    array_to_sort[i] = rand() % 30;
  }

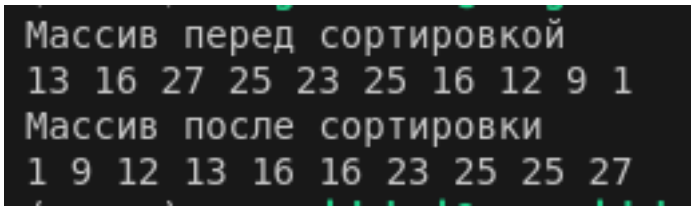
  printf("Массив перед сортировкой\n");
  print_array(array_to_sort, MAX_ARR_LEN);

  insert_sort(array_to_sort, MAX_ARR_LEN);

  printf("Массив после сортировки\n");
  print_array(array_to_sort, MAX_ARR_LEN);

  return 0;
}
```

Вывод:



Задание 2

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_LEN_ARR   25

void print_array(int* arr, int length)
{
  for(int i = 0; i < length; ++i) printf("%d ", arr[i]);
  putchar('\n');
}

void merge_sort(int* arr, int start, int mid, int end)
{
  int n1 = mid - start + 1;
  int n2 = end - mid;

  int L[n1], R[n2];

  for(int i = 0; i < n1; ++i) {
    L[i] = arr[start+i];
  }
```

```
    for(int i = 0; i < n2; ++i) {
        R[i] = arr[mid+1+i];
    }

    int i = 0, j = 0, k = start;
    while(i < n1 && j < n2) {
        if(L[i] < R[j]) {
            arr[k] = L[i];
            ++i;
        } else {
            arr[k] = R[j];
            ++j;
        }
        ++k;
    }

    while(i < n1) {
        arr[k] = L[i];
        ++i;
        ++k;
    }

    while(j < n2) {
        arr[k] = R[j];
        ++j;
        ++k;
    }
}

void recursive_merging(int* arr, int start, int end)
{
    if(start < end) {
        int mid = (start + end) / 2;

        recursive_merging(arr, start, mid);
        recursive_merging(arr, mid + 1, end);

        merge_sort(arr, start, mid, end);
    }
}

int main (void)
{
    int array_to_sort[MAX_LEN_ARR];

    for(int i = 0; i < MAX_LEN_ARR; ++i) {
```

```
    array_to_sort[i] = rand() % 30;
  }

  printf("Массив перед сортировкой\n");
  print_array(array_to_sort, MAX_LEN_ARR);

  recursive_merging(array_to_sort, 0, MAX_LEN_ARR-1);

  printf("Массив после сортировки\n");
  print_array(array_to_sort, MAX_LEN_ARR);

  return 0;
}
```

Вывод:

```
Массив перед сортировкой
13 16 27 25 23 25 16 12 9 1 2 7 20 19 23 16 0 6 22 16 11 8 27 9 2
Массив после сортировки
0 1 2 2 6 7 8 9 9 11 12 13 16 16 16 16 19 20 22 23 23 25 25 27 27
```

Задание 3

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_LEN_ARR    10

void print_array(int* arr, int length)
{
  for(int i = 0; i < length; ++i) printf("%d ", arr[i]);
  putchar('\n');
}

void quick_sort_support_last(int* arr, int start, int end)  // опорный элемент - последний элемент массива
{
  if(start < end) {
    int supporting_element_index = end;

    for(int i = start; i != supporting_element_index;) {
      if(arr[i] >= arr[supporting_element_index]) {
        int tmp = arr[i];
        arr[i] = arr[supporting_element_index-1];
        arr[supporting_element_index-1] = arr[supporting_element_index];

        arr[supporting_element_index] = tmp;
```

```
            --supporting_element_index;
            continue;
          }
          ++i;
        }

        quick_sort_support_last(arr, start, supporting_element_index-1);
        quick_sort_support_last(arr, supporting_element_index+1, end);
    }
}

void quick_sort_support_first(int* arr, int start, int end)  // опорный элемент - последний элемент массива
{
    if(start < end) {
        int supporting_element_index = start;

        for(int i = start+1; i <= end;) {
            if(arr[i] < arr[supporting_element_index]) {
                int tmp = arr[i];
                arr[i] = arr[supporting_element_index+1];
                arr[supporting_element_index+1] = arr[supporting_element_index];

                arr[supporting_element_index] = tmp;
                ++supporting_element_index;
                continue;
            }
            ++i;
        }

        quick_sort_support_first(arr, start, supporting_element_index-1);
        quick_sort_support_first(arr, supporting_element_index+1, end);
    }
}

int main (void)
{
    srand(time(NULL));
    int array_to_sort[MAX_LEN_ARR];

    for(int i = 0; i < MAX_LEN_ARR; ++i) {
        array_to_sort[i] = rand() % 30;
    }

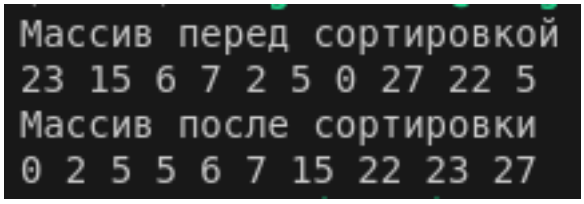
    printf("Массив перед сортировкой\n");
    print_array(array_to_sort, MAX_LEN_ARR);
```

```
    quick_sort_support_first(array_to_sort, 0, MAX_LEN_ARR-1);

    printf("Массив после сортировки\n");
    print_array(array_to_sort, MAX_LEN_ARR);

    return 0;
}
```

Вывод:



Задание 4

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define MAX_LEN_ARR    25

void quick_sort_support_last(int* arr, int start, int end)  // опорный элемент - последний элемент массива
{
    if(start < end) {
        int supporting_element_index = end;

        for(int i = start; i != supporting_element_index;) {
            if(arr[i] >= arr[supporting_element_index]) {
                int tmp = arr[i];
                arr[i] = arr[supporting_element_index-1];
                arr[supporting_element_index-1] = arr[supporting_element_index];

                arr[supporting_element_index] = tmp;
                --supporting_element_index;
                continue;
            }
            ++i;
        }

        quick_sort_support_last(arr, start, supporting_element_index-1);
        quick_sort_support_last(arr, supporting_element_index+1, end);
    }
```

```
}

void print_array_with_indexes(int* arr, int length)
{
   printf("Массив:  ");
   for(int i = 0; i < length; ++i) {
      printf("%3d ", arr[i]);
   }
   putchar('\n');

   printf("Индексы: ");
   for(int i = 0; i < length; ++i) {
      printf("%3d ", i);
   }
   putchar('\n');
}

int binary_search(int* arr, int el, int start, int end)
{
   if(start > end) return -1;
   int mid = (start + end) / 2;

   if(el == arr[mid]) return mid;
   else if(el > arr[mid]) return binary_search(arr, el, mid+1, end);
   else return binary_search(arr, el, start, mid);
}

int main(void)
{
   srand(time(NULL));
   int array_to_search[MAX_LEN_ARR] = {28,  57,  59,  72,  86, 105, 106, 129, 157, 203, 213, 219, 247, 252, 300, 339

   quick_sort_support_last(array_to_search, 0, MAX_LEN_ARR-1);

   int elem_to_search, index;

   printf("Массив, в котором нужно найти элементы:\n");
   print_array_with_indexes(array_to_search, MAX_LEN_ARR);

   printf("Результаты поиска (индекс -1 означает, что элемента нет в списке):\n");

   elem_to_search = 247;
   index = binary_search(array_to_search, elem_to_search, 0, MAX_LEN_ARR-1);
   printf("Индекс элемента %d: %d\n", elem_to_search, index);

   elem_to_search = 407;
```

```
    index = binary_search(array_to_search, elem_to_search, 0, MAX_LEN_ARR-1);
    printf("Индекс элемента %d: %d\n", elem_to_search, index);

    elem_to_search = 86;
    index = binary_search(array_to_search, elem_to_search, 0, MAX_LEN_ARR-1);
    printf("Индекс элемента %d: %d\n", elem_to_search, index);

    elem_to_search = 4654;
    index = binary_search(array_to_search, elem_to_search, 0, MAX_LEN_ARR-1);
    printf("Индекс элемента %d: %d\n", elem_to_search, index);

    elem_to_search = 444;
    index = binary_search(array_to_search, elem_to_search, 0, MAX_LEN_ARR-1);
    printf("Индекс элемента %d: %d\n", elem_to_search, index);

    return 0;
}
```

Вывод:

```
Массив, в котором нужно найти элементы:
Массив:   28  57  59  72  86 105 106 129 157 203 213 219 247 252 300 339 372 392 393 396 407 424 444 465 493
Индексы:   0   1   2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17  18  19  20  21  22  23  24
Результаты поиска (индекс -1 означает, что элемента нет в списке):
Индекс элемента 247: 12
Индекс элемента 407: 20
Индекс элемента 86: 4
Индекс элемента 4654: -1
Индекс элемента 444: 22
```

Задание 5

```
#include "sorting.h"
#include <stdio.h>
#include <time.h>
#include <stdlib.h>

#define MAX_LEN_ARR    100000

char is_sorted(int* arr, int length)
{
    for(int i = 0; i < length-1; ++i) {
        if(arr[i] > arr[i+1]) return 0;
    }
    return 1;
}

int main(void)
{
    int array[MAX_LEN_ARR];
```

```
for(int i = 0; i < MAX_LEN_ARR; ++i) {
    array[i] = rand() % 2000;
}

// 1. Вставки
clock_t current_time;
current_time = clock();

insert_sort(array, MAX_LEN_ARR);

current_time = clock() - current_time;

if(is_sorted(array, MAX_LEN_ARR) == 0) {
    printf("Массив не отсортивался.\n");
    return 1;
}

printf("Вставки - %lf секунд\n", (double)current_time / (double)CLOCKS_PER_SEC);

// обновление массива
for(int i = 0; i < MAX_LEN_ARR; ++i) {
    array[i] = rand() % 2000;
}

// 2. Слияние
current_time = clock();

recursive_merging(array, 0, MAX_LEN_ARR-1);

current_time = clock() - current_time;

if(is_sorted(array, MAX_LEN_ARR) == 0) {
    printf("Массив не отсортивался.\n");
    return 1;
}

printf("Слияние - %lf секунд\n", (double)current_time / (double)CLOCKS_PER_SEC);

// обновление массива
for(int i = 0; i < MAX_LEN_ARR; ++i) {
    array[i] = rand() % 2000;
}

// 3. Быстрая, опорный - последний
current_time = clock();
```

```
    quick_sort_support_last(array, 0, MAX_LEN_ARR-1);

    current_time = clock() - current_time;

    if(is_sorted(array, MAX_LEN_ARR) == 0) {
        printf("Массив не отсортивался.\n");
        return 1;
    }

    printf("Быстрая, опорный последний - %lf секунд\n", (double)current_time / (double)CLOCKS_PER_SEC);

    // обновление массива
    for(int i = 0; i < MAX_LEN_ARR; ++i) {
        array[i] = rand() % 2000;
    }

    // 4. Быстрая, опорный - первый
    current_time = clock();

    quick_sort_support_first(array, 0, MAX_LEN_ARR-1);

    current_time = clock() - current_time;

    if(is_sorted(array, MAX_LEN_ARR) == 0) {
        printf("Массив не отсортивался.\n");
        return 1;
    }

    printf("Быстрая, опорный первый - %lf секунд\n", (double)current_time / (double)CLOCKS_PER_SEC);

    return 0;
}
```

Вывод:

```
Вставки - 9.044492 секунд
Слияние - 0.012853 секунд
Быстрая, опорный последний - 0.019453 секунд
Быстрая, опорный первый - 0.014594 секунд
```